

Aula 008

1) Python

a)

```
main.py x + Console
1 def adicionar(item, lista=[]):
2     lista.append(item)
3     return lista
4
5 print(adicionar(1))
6 print(adicionar(2))
```

[1]
[1, 2]

[1] e [1, 2]

b) Valores padrão em parâmetros mutáveis: a lista padrão é criada uma vez só e reaproveitada entre as chamadas.

2) Java

a)

```
J Main.java x + Console
1 public class Main {
2     static void zera(int[] v, int n) {
3         v[0] = 0;
4         n = 0;
5     }
6
7     public static void main(String[] args) {
8         int[] v = {5, 5}; int n = 5;
9         zera(v, n);
10        System.out.println(v[0] + " " + n);
11    }
12 }
```

0 5

0 5

b) Passagem por valor: o n é copiado, então não muda. Do array, o que é copiado é a referência, que aponta para o mesmo array, por isso v[0] muda.

3) Python

a)

```
main.py x + Console
1 fs = [lambda: i for i in range(3)]
2 print([f() for f in fs])
```

[2, 2, 2]

[2, 2, 2]

b) Fechamentos: as lambdas capturam a variável i, e não o valor; quando são chamadas, o laço já terminou e i vale 2.

4) C

a)

```
C Main.c x +
1 #include <stdio.h>
2
3 int contador(void) {
4     static int n = 0;
5     return ++n;
6 }
7
8 int main(void) {
9     contador(); contador();
10    printf("%d\n", contador());
11    return 0;
12 }
```

Console

3

3

b) Variáveis locais estáticas: n é inicializado uma vez e mantém o valor entre as chamadas.

5) Rust

a)

```
HelloWorld.rs x Console I/O AI Agent Exit 1
1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7     let d = dobra(v);
8     println!("{:?} {:?}", v, d);
9 }
```

error[E0382]: borrow of moved value: `v`
--> HelloWorld.rs:8:27

6 | let v = vec![1, 2, 3];
| - move occurs because `v` has type `Vec<i32>`, which does not implement the `Copy` trait
7 | let d = dobra(v);
| - value moved here
8 | println!("{:?} {:?}", v, d);
| ^ value borrowed here after move

note: consider changing this parameter type in function `dobra` to borrow instead if owning the value isn't necessary
--> HelloWorld.rs:1:13

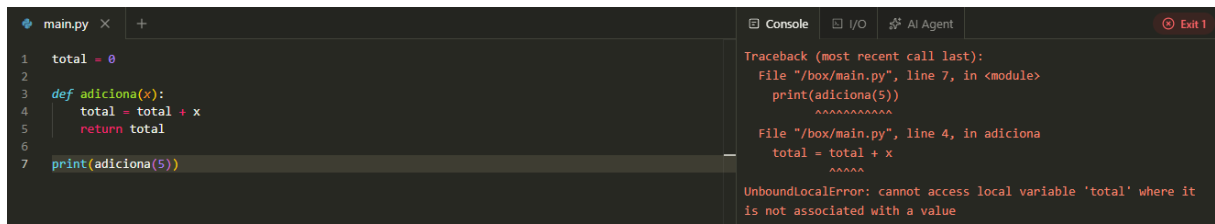
1 | fn dobra(v: Vec<i32>) -> Vec<i32> {
| ----- ^^^^^^^^^ this parameter takes ownership of the value
| |
| in this function
help: consider cloning the value if the performance cost is acceptable
7 | let d = dobra(v.clone());
| ++++++

error[E0382]: borrow of moved value: `v`

b) Posse (ownership): dobra(v) recebe o Vec por valor, então a posse de v é movida e ele não pode ser usado depois.

6) Python

a)



The screenshot shows a Python IDE with a file named 'main.py' and a console window. The code in 'main.py' is as follows:

```
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5     return total
6
7 print(adiciona(5))
```

The console window displays a traceback for an `UnboundLocalError`:

```
Traceback (most recent call last):
  File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ~~~~~
  File "/box/main.py", line 4, in adiciona
    total = total + x
    ~~~~~
UnboundLocalError: cannot access local variable 'total' where it
is not associated with a value
```

UnboundLocalError

b) Escopo local vs. externo: como total recebe atribuição dentro da função, ele é tratado como local e é lido antes de ter valor.